



Exceptional service in the national interest

AN OVERVIEW OF THE TRILINOS CORE PRODUCTS

Roger Pawlowski

HPSF Conference 2026

March 16-20, 2025, Chicago, IL



Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525.

SAND2026-18416C

CORE AREA PROVIDES FOUNDATIONAL CAPABILITIES FOR PACKAGES



- Performant data structures
- Abstraction layers (for linear algebra, linear solvers and nonlinear analysis)
- Utility libraries
- Common look and feel
- Includes “snapshotted” packages: Kokkos, Kokkos-Kernels and SEACAS

CORE AREA – PROVIDES BASIC CAPABILITIES



Alegbra/Numerics		
Kokkos (@kokkos)	Performance portability library	Christian Trott (@crtrott)
Kokkos Kernels (@kokkos-kernels)	Optimized math kernels for Kokkos library	Luc Berger-Vergiat (@lucbv)
Tpetra (@tpetra)	Distributed linear algebra objects	Chris Siefert (@csiefert2)
RTOp (@rtop)	Vector Reduction/Transformation Operators	Ross Bartlett (@bartlettroscoe)
Thyra (@thyra)	Abstract numerical algorithms and interfaces	Ross Bartlett (@bartlettroscoe)
Utilities		
Galeri (@galeri)	Matrix generation tools	Jonathan Hu (@jhux2)
PyTrilinos2 (@pytrilinos2)	Python bindings for Trilinos packages	Christian Glusa (@cgcgcg)
SEACAS (@SEACAS)	Utilities supporting finite element analysis	Tolu Okusanya (@tokusanya)
Teuchos (@teuchos)	Common tools (<i>smart pointers, parameter lists, ...</i>)	Roger Pawlowski (@rppawlo)
Meshing / Domain Decomposition		
Pamgen (@pamgen)	Mesh generation tools for simple shapes	Roger Pawlowski (@rppawlo)
Zoltan/Zoltan2 (@zoltan, @zoltan2)	Load balancing and partitioning	Erik Boman (@egboman)

TEUCHOS: COMMON PACKAGE UTILITIES (PRONOUNCED “TEFOS”)



- Provides a common look and feel
- Capabilities:
 - Reference counted smart pointer (**RCP**) – interoperable with `std::shared_ptr`
 - **ParameterList**: runtime database (with read/write support for yaml and xml)
 - C++ Abstraction for an **MPI**
 - **Timers: TimeMonitor and StackedTimer**
 - CPU BLAS/LAPACK wrappers
 - Scalar traits
 - Unit test harness
 - I/O Interface
 - Serialization

TEUCHOS: PARAMETER LIST EXAMPLE

- ParameterList is a runtime database of C++ objects
- Common look and feel across packages
- Many packages provide factories/ctors that use PLists
- Can Read/Write to:
 - XML
 - YAML
- Most packages validate the parameter lists

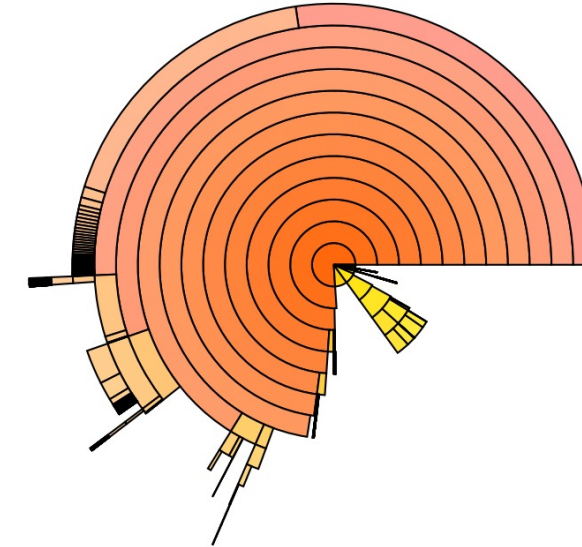
```
Stratimikos:
  Linear Solver Type: Belos
  Linear Solver Types:
    Belos:
      Solver Type: Block GMRES
      Solver Types:
        Block GMRES:
          Convergence Tolerance: 1.0e-05
          Output Frequency: 10
          Output Style: 1
          Verbosity: 33
          Maximum Iterations: 100
          Block Size: 1
          Num Blocks: 20
          Flexible Gmres: false
      Preconditioner Type: Ifpack2
      Preconditioner Types:
        Ifpack2:
          Overlap: 1
          Prec Type: ILUT
          Ifpack2 Settings:
            'fact: drop tolerance': 0.0
            'fact: ilut level-of-fill': 1.0
            'fact: level-of-fill': 1
```

```
Teuchos::RCP<Teuchos::ParameterList> p = Teuchos::parameterList();
p->set("Linear Solver Type", "Belos");
Teuchos::ParameterList& belosList = p->sublist("Linear Solver Types").sublist("Belos");
belosList.set("Solver Type", "Pseudo Block GMRES");
belosList.sublist("Solver Types").sublist("Pseudo Block GMRES").set<int>("Maximum Iterations", 200);
belosList.sublist("Solver Types").sublist("Pseudo Block GMRES").set<int>("Num Blocks", 200);
belosList.sublist("Solver Types").sublist("Pseudo Block GMRES").set("Verbosity", 0x7f);
belosList.sublist("Solver Types").sublist("Pseudo Block GMRES").set("Output Frequency", 100);
belosList.sublist("VerboseObject").set("Verbosity Level", "none");
p->set("Preconditioner Type", "Ifpack2");
Teuchos::ParameterList& ifpackList = p->sublist("Preconditioner Types").sublist("Ifpack2");
ifpackList.set("Prec Type", "ILUT");
ifpackList.set("Overlap", 3);
ifpackList.sublist("Ifpack2 Settings").set("fact: ilut level-of-fill", 2.0);
```



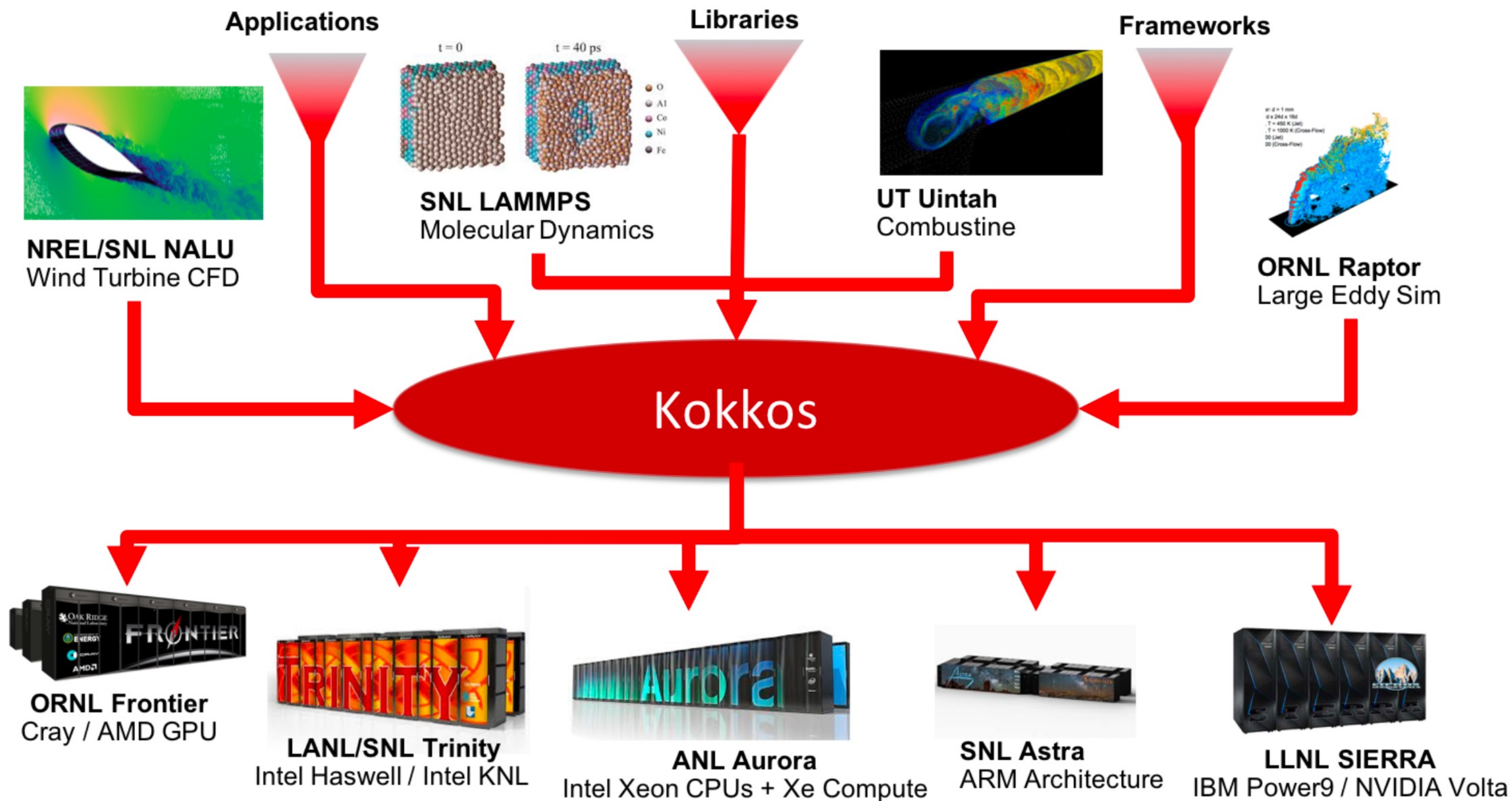
TIMERS

- **Teuchos::TimeMonitor** is the main timer used by Trilinos packages
 - Destructor stops timer: exception safe – excellent for libraries
- **Teuchos::StackedTimer** is a new impl that gives more information
 - Explicit start/stop: NOT exception safe – easy to read for apps teams
- TimeMonitor injects values into StackedTimer
- Trilinos developers:
 - **Always use TimeMonitor in your libraries to start/stop timers**
- Trilinos Users:
 - **Always use StackedTimer to print your data**



```
Panzer MixedPoisson Test: 4.39393 [1]
|   Mixed Poisson: 4.39382 - 99.9976% [1]
|   |   panzer::CubeHexMeshFactory::buildUncomittedMesh(): 0.000528984 - 0.0120393% [1]
|   |   panzer::CubeHexMeshFactory::completeMeshConstruction(): 0.765863 - 17.4305% [1]
|   |   panzer::DOFManagerFactory::buildUnqueGlobalIndexer: 0.0451156 - 1.0268% [1]
...
|   |   Ifpack2::Relaxation::initialize: 4.403e-06 - 0.000100209% [1]
|   |   Ifpack2::Relaxation::compute: 0.00249703 - 0.0568305% [1]
|   |   Belos: Operation Op*x: 0.00338377 - 0.0770121% [1]
|   |   Belos: PseudoBlockGmresSolMgr total solve time: 0.432027 - 9.83259% [1]
|   |   |   Belos: ICGS[2]: Orthogonalization: 0.0212883 - 4.92755% [31]
|   |   |   |   Belos: ICGS[2]: Ortho (Norm): 0.000489003 - 2.29705% [31]
|   |   |   |   Belos: ICGS[2]: Ortho (Inner Product): 0.00904903 - 42.507% [60]
|   |   |   |   Belos: ICGS[2]: Ortho (Update): 0.00662776 - 31.1333% [60]
|   |   |   |   Remainder: 0.00512255 - 24.0627%
|   |   |   Belos: Operation Prec*x: 0.0420421 - 9.73136% [31]
|   |   |   |   Ifpack2::Relaxation::apply: 0.0418983 - 99.658% [31]
|   |   |   |   Remainder: 0.000143774 - 0.341977%
|   |   |   Belos: Operation Op*x: 0.104452 - 24.1772% [31]
|   |   |   Remainder: 0.264244 - 61.1639%
...
```

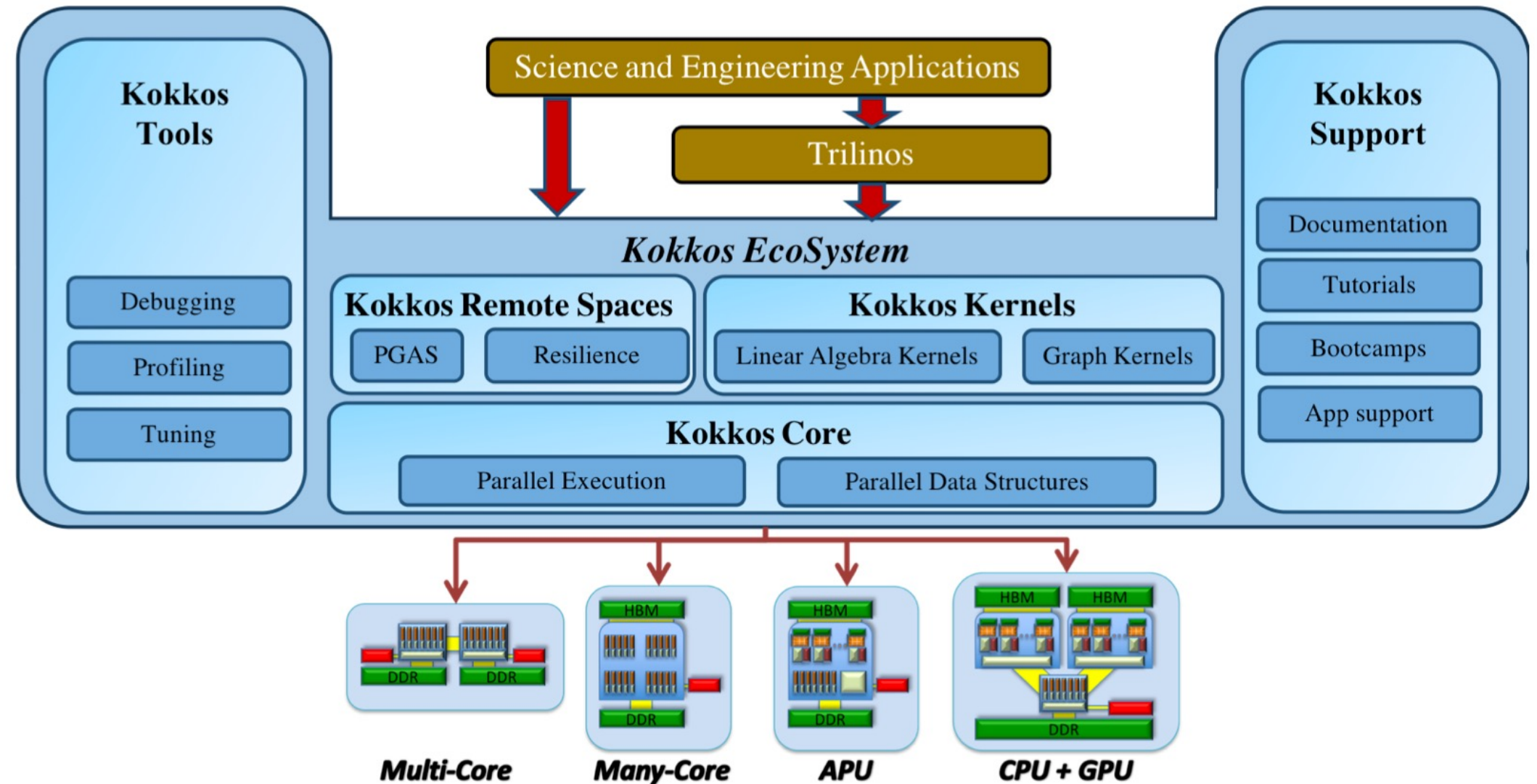
PERFORMANCE PORTABILITY: KOKKOS



PERFORMANCE PORTABILITY: KOKKOS™



- Shared memory programming model in C++
- Write once, run on many architectures
- Thread scalable parallel patterns: for, reduce, scan
- Execution and Memory spaces
- Multidimensional array with compile-time polymorphic layouts (row major vs column major)
- Kokkos devs are on the c++ committee



<https://github.com/kokkos/kokkos>

EXAMPLE: PARALLEL SUM

- Kokkos must be initialized and finalized (once!)
- Rank 2 View, with one run-time (*) and one compile-time dimension
- parallel_reduce pattern
- Lambda creates a functor used in the execution space
- Can explicitly force Execution Space, Memory Space and View Layout (this example uses the defaults)

```
#include <Kokkos_Core.hpp>
#include <cstdio>

using view_type = Kokkos::View<double* [3]>;

int main(int argc, char* argv[]) {
    Kokkos::initialize(argc, argv);

    {
        view_type a("A", 10);

        Kokkos::parallel_for(
            10, KOKKOS_LAMBDA(const int i) {
                a(i, 0) = 1.0 * i;
                a(i, 1) = 1.0 * i * i;
                a(i, 2) = 1.0 * i * i * i;
            });

        double sum = 0;
        Kokkos::parallel_reduce(
            10,
            KOKKOS_LAMBDA(const int i, double& lsum) {
                lsum += a(i, 0) * a(i, 1) / (a(i, 2) + 0.1);
            },
            sum);

        printf("Result: %f\n", sum);
    }
    Kokkos::finalize();
}
```

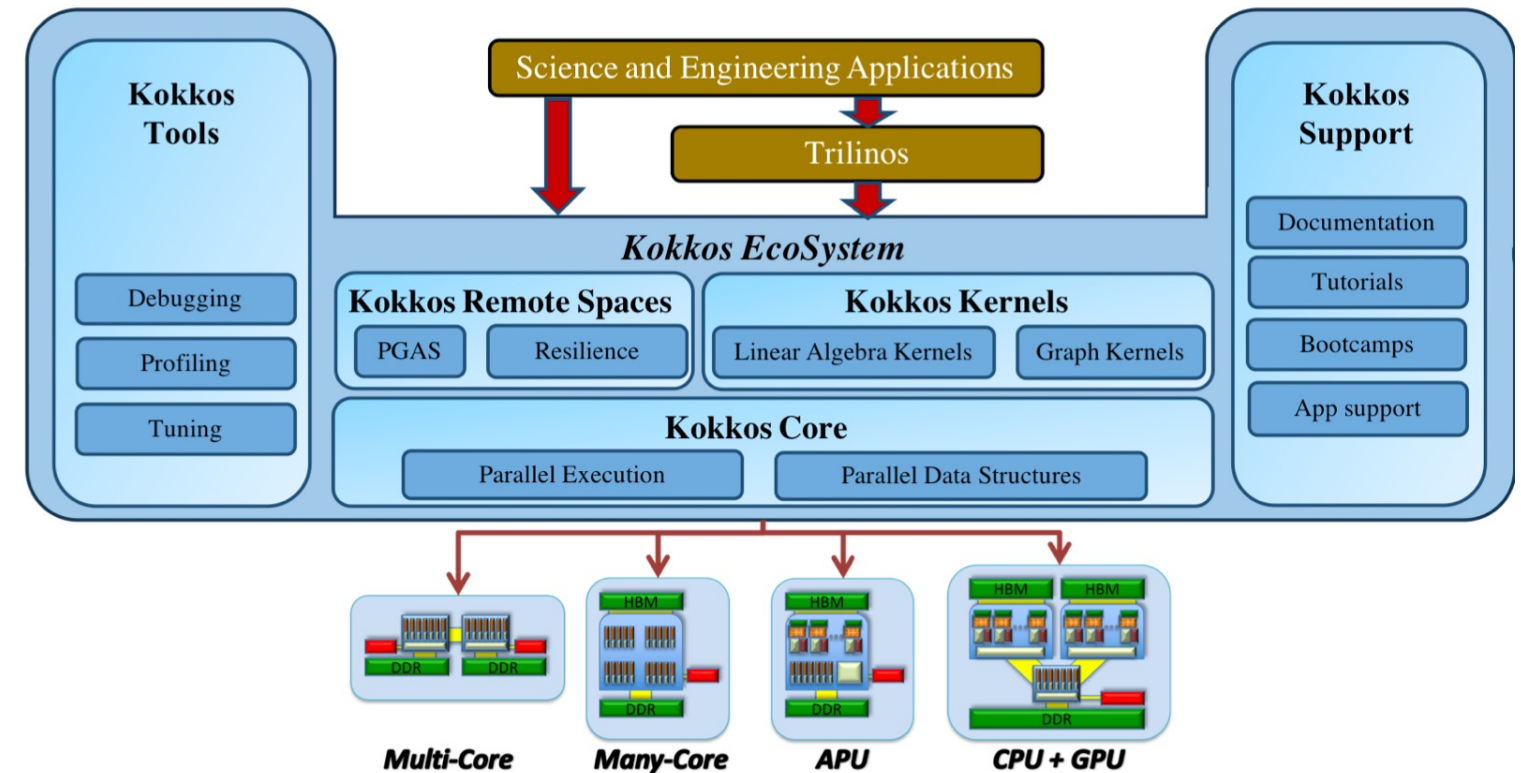


PERFORMANCE PORTABLE LINEAR ALGEBRA: KOKKOS-KERNELS



KokkosKernels provides math kernels for dense and sparse linear algebra as well as graph computations.

- Portable BLAS, Sparse and Graph kernels
- Generic implementations for various scalar types and data layouts
- Access to major vendor optimized math libraries (rocblas, cublas, ...)
- Expand the scope of BLAS to hierarchical implementations.
- Batched BLAS



PARALLEL SPARSE LINEAR ALGEBRA: TPETRA

Basic building blocks for Trilinos solvers and preconditioners for parallel distributed architectures

- **Provides**

- Maps (distributed index array)
- Sparse graphs, (block) sparse matrices, & dense vectors/multivectors
- Norms, SpMV, SpGEMM, ...
- MPI communication & data (re)distribution
- Fill: {Create,modify} {graph,matrix,vector}

- **Key features**

- Can pick type of values (real, complex, automatic differentiation, ensemble, ...)
- Detailed performance monitoring (**see Chris Siefert's talk on 3/19 @10:45am**)



Image source:
https://en.wikipedia.org/wiki/Siq#/media/File:Petra_First_Glimpse.jpg



TPETRA OVERVIEW

- **Map:** Global indices on each process
- **Graph:** Non-zero entries in a Sparse graph
- **CrsMatrix:** sparse distributed matrix
- **FECrsGraph/FECrsMatrix:** helper classes to assemble finite element matrices from element-wise distributed mesh
- **Vector:** Dense distributed vector
- **MultiVector:** Multiple columns of dense distributed vectors
- **Import/Export:** data movement

Case 2: Twist 1

- ♦ First 2 rows of A , first element of y and last 2 elements of x , kept on PE 0.
- ♦ Last row of A , last 2 element of y and first element of x , kept on PE 1.

PE 0 Contents

$$y = [y_1], \dots A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \end{bmatrix}, \dots x = \begin{bmatrix} x_2 \\ x_3 \end{bmatrix}$$

- RowMap = {0, 1}
- ColMap = {0, 1, 2}
- DomainMap = {1, 2}
- RangeMap = {0}

PE 1 Contents

$$y = \begin{bmatrix} y_2 \\ y_3 \end{bmatrix}, \dots A = [0 \quad -1 \quad 2], \dots x = [x_1]$$

- RowMap = {2}
- ColMap = {1, 2}
- DomainMap = {0}
- RangeMap = {1, 2}

Original Problem

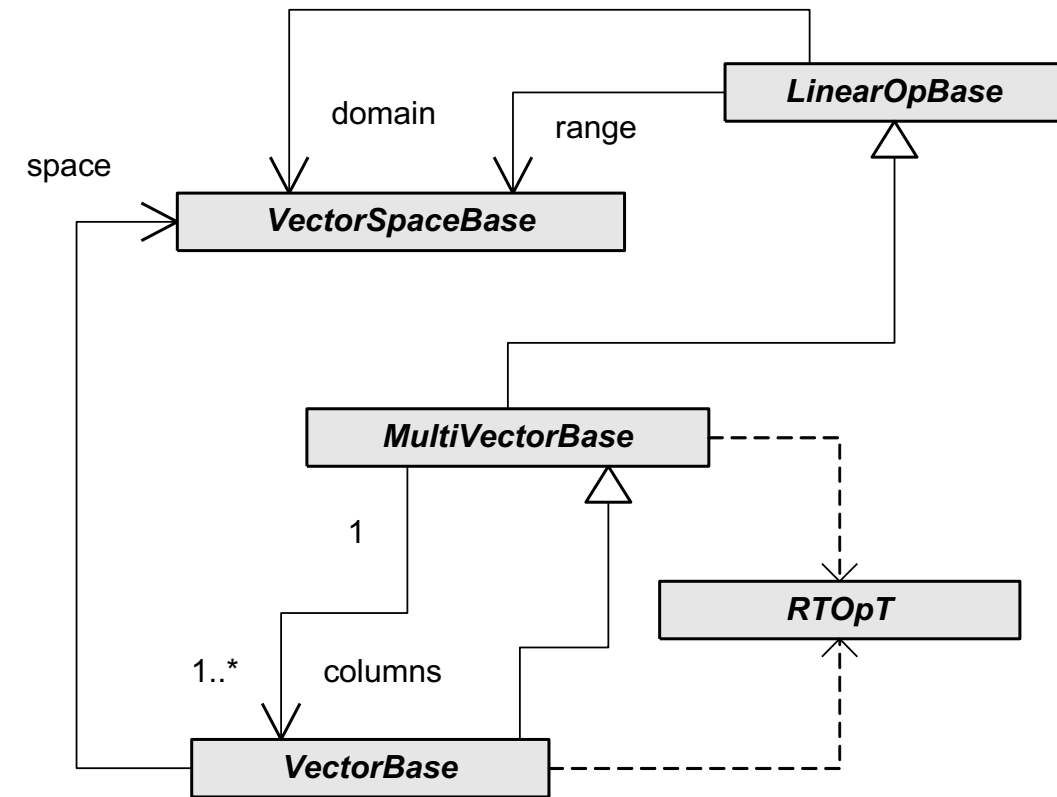
$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Notes:

- Rows are wholly owned.
- RowMap is NOT = DomainMap
is NOT = RangeMap (all 1-to-1).
- ColMap is NOT 1-to-1.
- Call to FillComplete:
A.FillComplete(DomainMap, RangeMap);

SOLVER ABSTRACTION LAYER: THYRA

- Abstractions for:
 - Vector/Operator (+Blocked wrappers)
 - Linear solvers/Preconditioners
 - Nonlinear solvers
 - Physics model interface (solver callback API)
- Matrix/Vector operations are handled in native data structures
 - Tpetra, PETSC, Sundials, CPU BLAS/LAPACK, ...
- Used to write higher level packages/algorithms
 - Wrapper to all linear solver/preconditioners (**Stratimikos**)
 - Physics-based block preconditioners (**Teko**)
 - Nonlinear solver (**NOX**)
 - Time integration (**Tempus**)
 - Optimization (**ROL**)
 - Bifurcation/Stability Analysis (**LOCA**)



$$\begin{bmatrix} Q_B & K_E^B & 0 \\ K_B^E & Q_E & Q_F^E \\ Q_B^F & Q_E^F & D_F \end{bmatrix} \begin{bmatrix} B \\ E \\ F \end{bmatrix}$$

GALERI

- Inline matrix generator
- Used for unit testing linear solvers and preconditioners
- Generates linear systems from different discretizations (FD/FE) and PDE operators
 - Cartesian 1D, 2D, 3D blocks
 - Laplace, Hilbert, Biharmonic, Cauchy, Fielder, ...

```
#define GO long long
#define Scalar int
#define LO int
#define Node Tpetra::KokkosClassic::DefaultNode::DefaultNodeType

using namespace Galeri;

int main(int argc, char *argv[]) {
    typedef Tpetra::Map<LO, GO, Node> Tpetra_Map;
    typedef Tpetra::CrsMatrix<Scalar, LO, GO, Node> Tpetra_CrsMatrix;

#ifdef HAVE_MPI
    MPI_Init(&argc, &argv);
#endif

    RCP<const Teuchos::Comm<int>> comm = Teuchos::DefaultComm<int>::getComm();

    std::string mapType = "Cartesian2D";
    auto mapParameters = Teuchos::ParameterList("Tpetra::Map");
    int nx = 5 * comm->getSize();
    int ny = 5 * comm->getSize();
    mapParameters.set("nx", nx);
    mapParameters.set("ny", ny);

    auto map = RCP{Galeri::Xpetra::CreateMap<Scalar, GO, Tpetra_Map>(mapType, comm, mapParameters)};

    auto matrix = Galeri::Xpetra::BigStar2D<Scalar, LO, GO, Tpetra_Map, Tpetra_CrsMatrix>
        (map, nx, ny, 20, -8, -8, -8, -8, 2, 2, 2, 2, 1, 1, 1, 1);

    ...

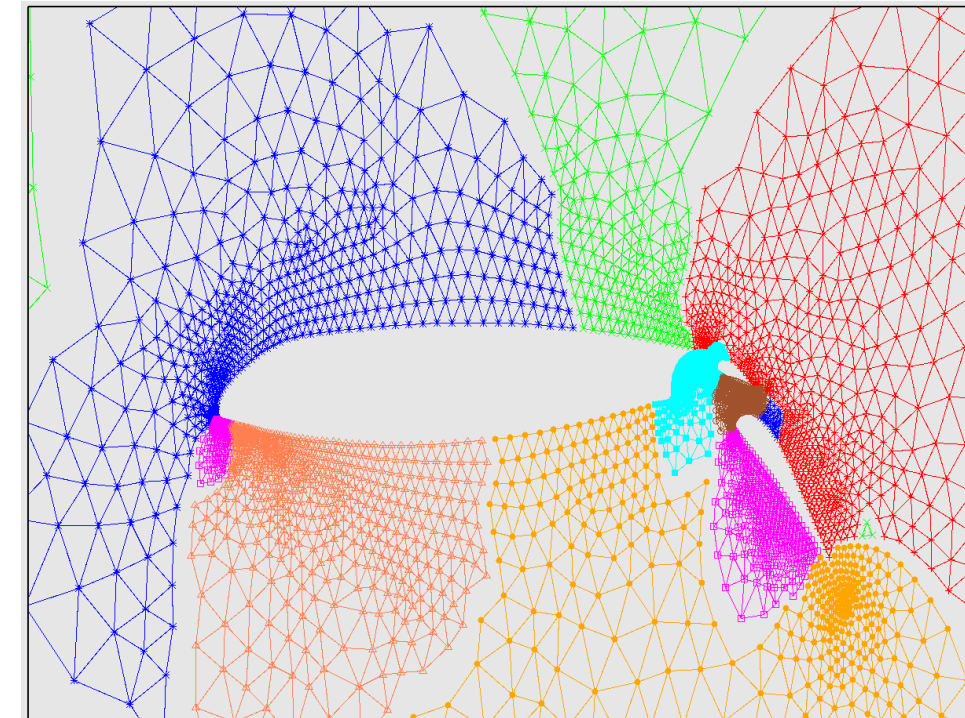
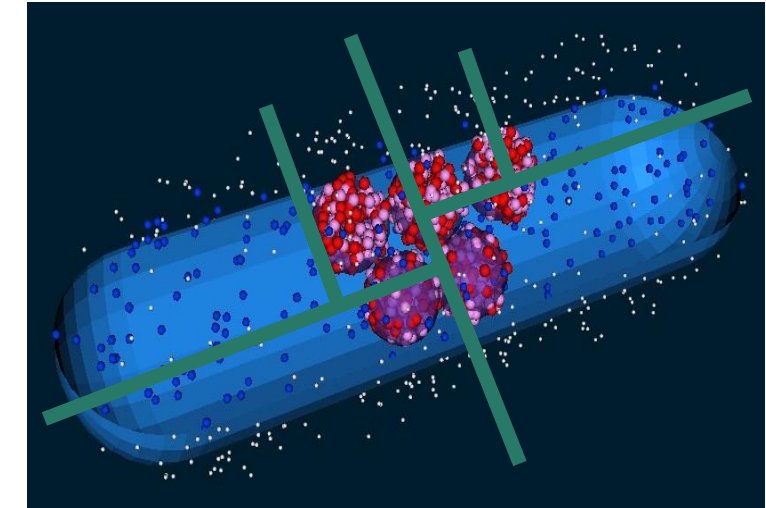
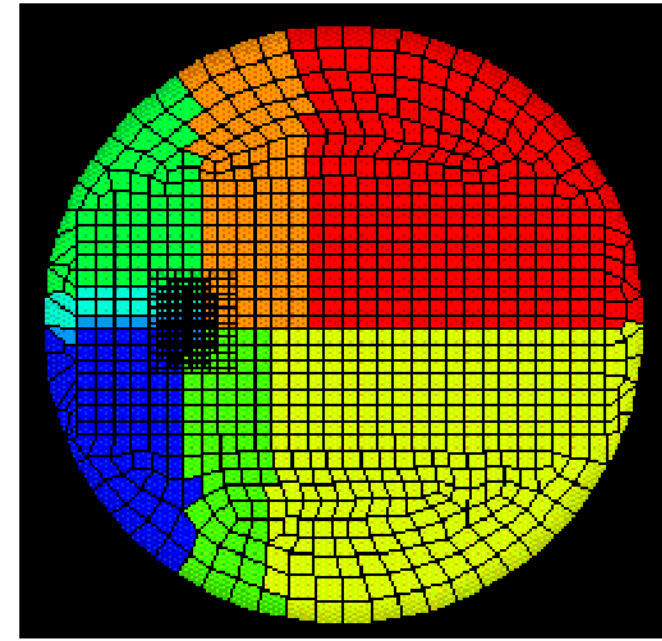
#ifdef HAVE_MPI
    MPI_Finalize();
#endif

    return (EXIT_SUCCESS);
}
```

PARALLEL PARTITIONING AND LOAD BALANCING: ZOLTAN/ZOLTAN2



- **Zoltan** and **Zoltan2** are packages for partitioning, load balancing, and combinatorial scientific computing
 - Both have a collection of algorithms (geometric partitioning, graph, hypergraph, etc.)
- Zoltan is quite old (~2000) and can be used stand-alone on CPUs
- Zoltan2 is fairly new and a tightly integrated Trilinos package
 - Supports Kokkos and GPUs
 - Works with Tpetra matrices and vectors
 - Currently depends on Zoltan (though some algorithms have been reimplemented)
 - Has similar features/functionality to Zoltan but not exactly the same



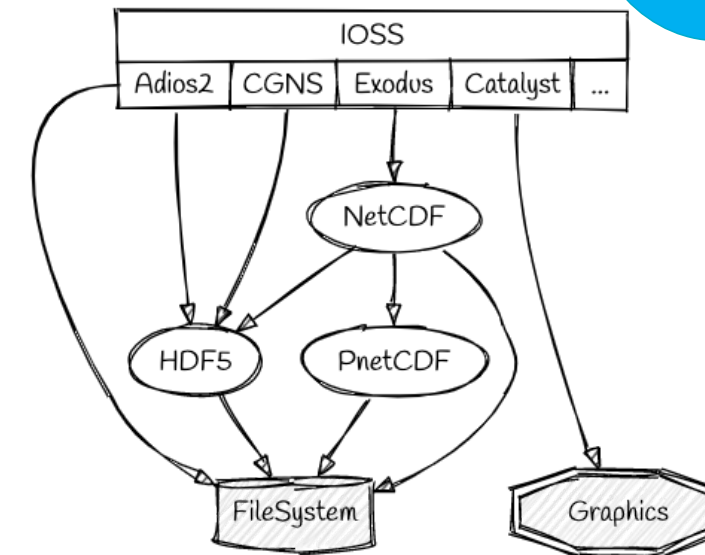
FINITE ELEMENT I/O TOOLS: SEACAS

- Provide IO for Sandia finite-element applications
- Provide applications and libraries for analyst workflow
- Support Production Analysts and Path-Forward Developers

SEACAS contains the **Exodus** and **IOSS** finite element input/output libraries; applications supporting meshing, preprocessing, postprocessing, graphics, database translation; and support libraries..

Exodus is a model developed to store and retrieve data for finite element analyses in a random access, machine independent, binary file that is written and read via C, C++, or Fortran APIs.

The **IO Subsystem** (IOSS) is a high-level database access API designed to give a format-agnostic interface to I/O capabilities with multiple backends outputting data to multiple formats. Supported input and output formats are **Exodus**, **CGNS**, HeartBeat, History, Generated, Pamgen, TextMesh, ADIOS2, FAODEL, Catalyst2, Null.



aprepro, algebra, blot, conjoin, cpup, ejoin, epu, exo2mat, exo_format, exodiff, exomatlab, explore fastq, gen3d, genshell, gjoin, grepos, io_shell, io_info, io_modify, mapvar, mat2exo, nas2exo, nem_slice, nem_spread, numbers, exotxt, txtexo, slice, zellij

Aprepro

loss

Zoltan

Exodus

ParMetis

NetCDF

MatIO

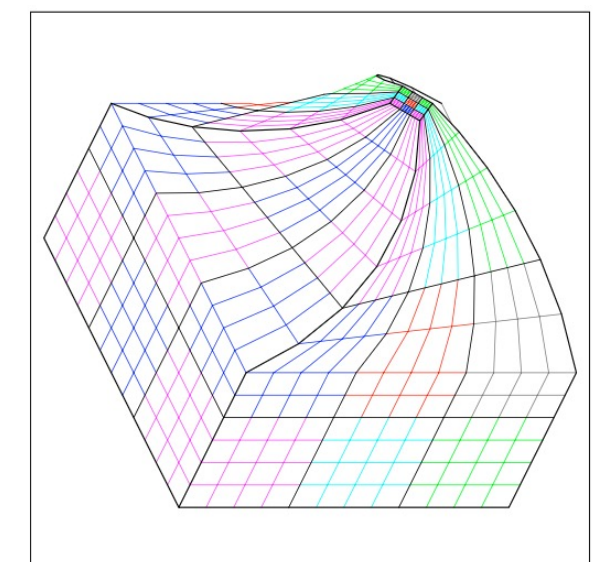
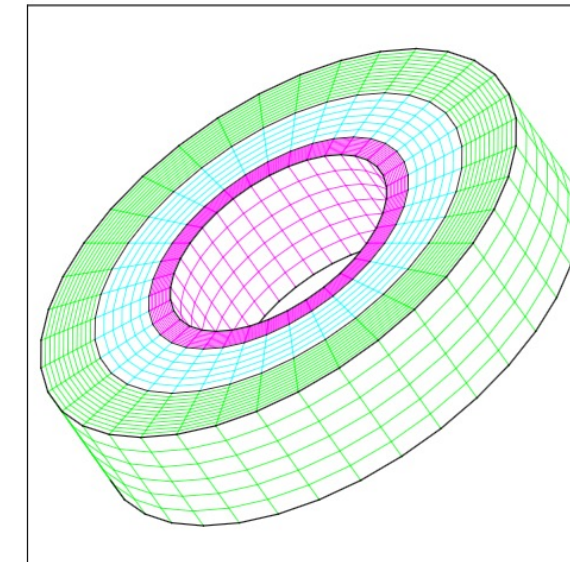
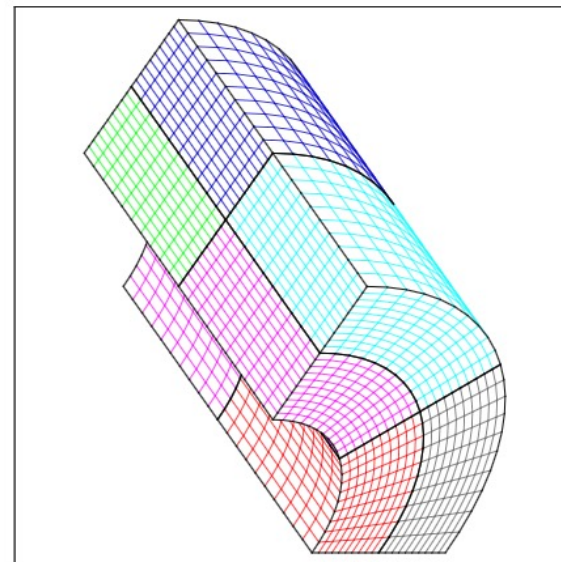
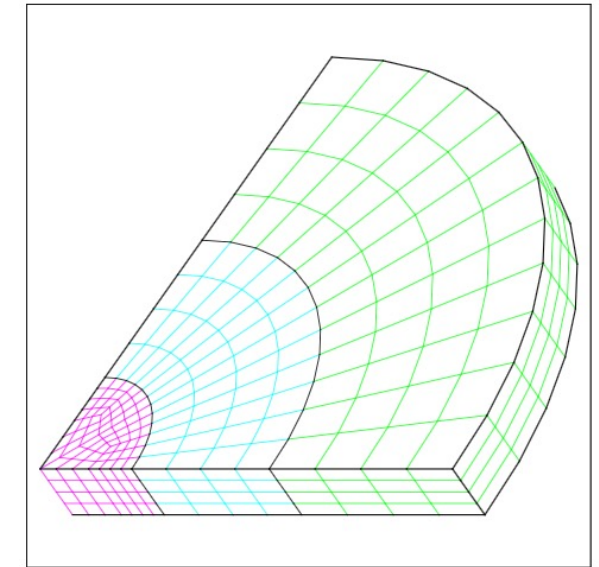
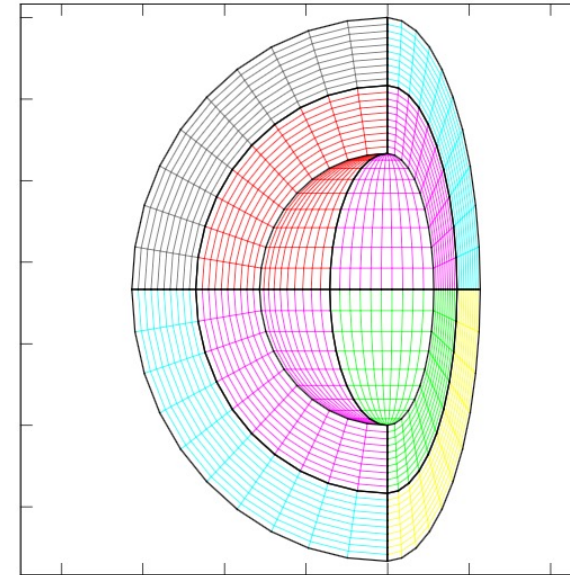
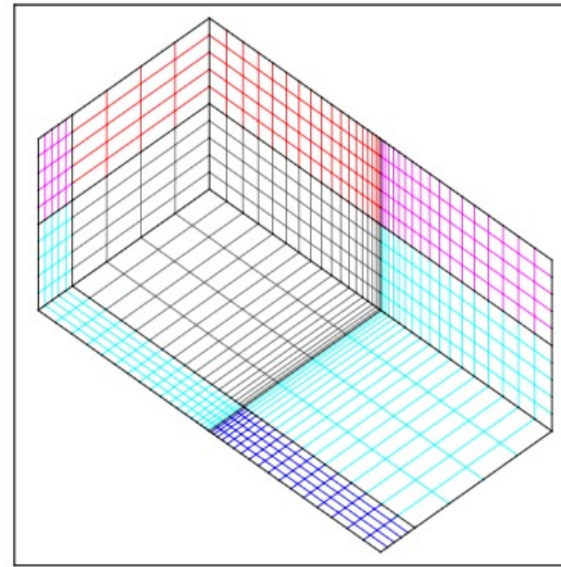
HDF5

PNetCDF

CGNS

A supercomputer is a device for turning compute-bound problems into I/O-bound problems

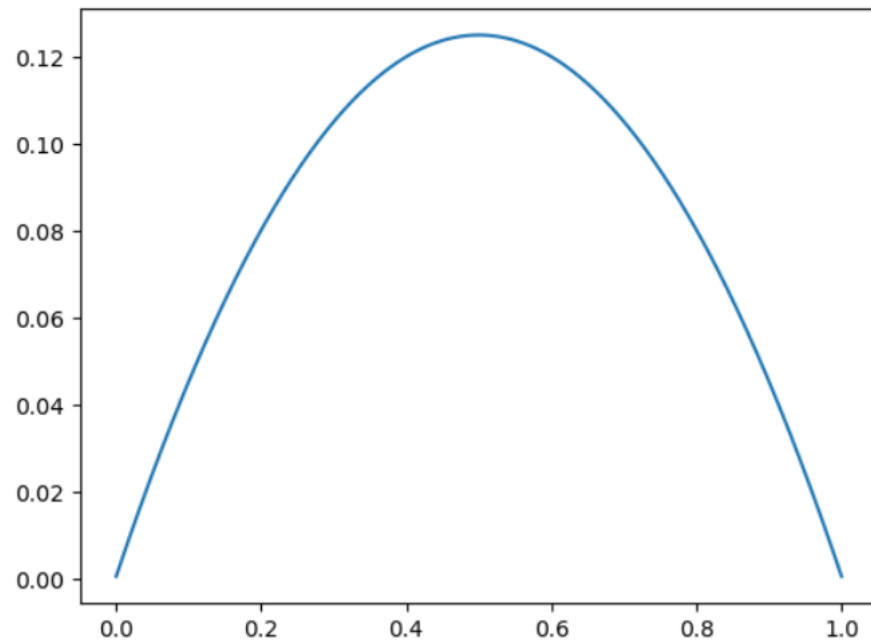
- Inline finite element mesh generator
- Hex and quads of simple shapes (cubes, spheres and cylinders)
- Supports geometry transformations
- Integrated into SEACAS, can populate a STK mesh
- Older package that is CPU only



“Pamgen, a Library for Parallel Generation of Simple Finite Element Meshes,”
Hensinger, et. al., Sand Report, SAND2008-1933, April 2003.

PYTRILINOS2

- Python wrappers for Trilinos
- Only wraps a subset of packages
 - Teuchos
 - Tpetra
 - MueLu
 - Stratimikos
- Requires pybind11 and binder (llvm)
- **Christian Glusa will have a tutorial at 4:20pm today!**



```
def main():
    comm = Teuchos.getTeuchosComm(MPI.COMM_WORLD)
    rank = comm.getRank()

    vectorType = Tpetra.Vector()

    n = 300000

    A = assemble1DLaplacian(n, comm)
    mapT = A.getRowMap()

    x = vectorType(mapT, True)
    b = vectorType(mapT, False)
    residual = vectorType(mapT, False)

    b.putScalar(1.)

    p = Teuchos.ParameterList()
    P = MueLu.CreateTpetraPreconditioner(A, p)

    x.putScalar(0.)
    norm_x = x.norm2()
    its = CG(A, x, b, max_iter=30, prec=P)
    norm_x = x.norm2()
    A.apply(x, residual)
    residual.update(1, b, -1)
    resNorm = residual.norm2()
```





END